

AI Architect Program Season 2:

OpenClaw Agents

Monday, April 27, 10:30 AM (PT)/1:30 PM (ET) 90 MIN



Tiffanie Kong

Senior Product Manager | AI
Platforms & Infrastructure



Rodrigo Hernández

IoT, ML & AI Consultant | Automation
& AI



Yong Tian

CTO & Co-Founder 10xjobs.co | HPC,
Cloud & B2B AI Workloads



AI Architect Program Season 2

Building and Deploying Secure OpenClaw Agents

6 week Program: Fundamentals of designing, securing and deploying secure OpenClaw agents.

01

Weeks 1–2: Foundations

Intro to Agent Architecture, OpenClaw Agents, Security and access permission restrictions.

02

Weeks 3–4: Hands on Build Labs

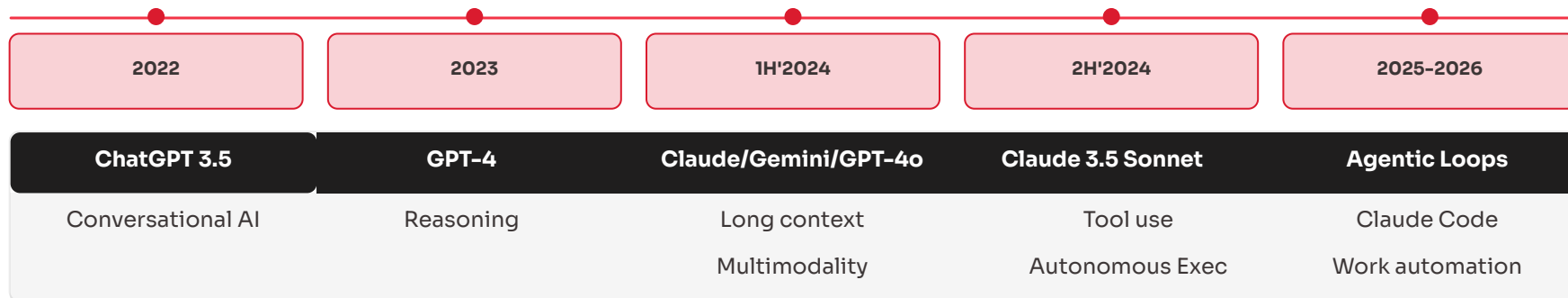
Group breakout sessions: setup environment and build 4 basic OpenClaw agents

03

Weeks 5–6: Lightning Talk & Community Showcase

Hear from OpenClaw experts on best practices for monitoring, testing, response, and recovery and when to hit the "kill switch". Wrap up program with members showcasing OpenClaw agents in action.

The Agentic AI Shift



LLMs

Text in, text out. Stateless — no memory between calls, no tools, no awareness of the world outside the prompt. Powerful reasoning with no hands.

Chatbots

A conversational UI wrapped around an LLM. Feels interactive but still reactive — waits for your next message, forgets everything when the session ends.

AI Agents

LLM + memory + tools + an execution loop. The agent perceives, reasons, acts, observes the result, and repeats — autonomously, until the task is done.

Always On Personal Agents

Persistent, self-hosted agents running 24/7 on your own hardware — managing your work and life across every app you use, while you sleep.

Software Evolution



Traditional Software	Chatbot	AI Agent	Always-On Agent	OpenClaw Agent
Slack, MS Office, VS Code	ChatGpt, Claude, Gemini	Cursor, Lovable, Google AI Studio	Claude Code, OpenClaw, 10xJobs	Claude Code, OpenClaw, 10xJobs
- Narrow focus - Prescribed workflow - Structured data format - Reactive	- General purpose - Understand user intent - Unstructured input - Adaptive solution - Description only - No memory (original) - Reactive	- Narrow focus - Understand user intent - Unstructured input - Adaptive solution - Reactive	- General purpose - Understand user intent - Unstructured input - Adaptive solution - Implement solution w/ tools - Previous interaction memory - Proactive	- General purpose - Understand user intent - Unstructured input - Adaptive solution - Implement solution w/ tools - Previous interaction memory - Proactive

Agent Building Blocks & Behavior

Components of an AI agent



LLM (Brain)

Core reasoning engine. understands instructions, reasons, plans, makes decisions



Memory (Context)

Stores/Retrieves data, RAG, past discussions. Provides short term (session context), long-term (vector store/file) to model

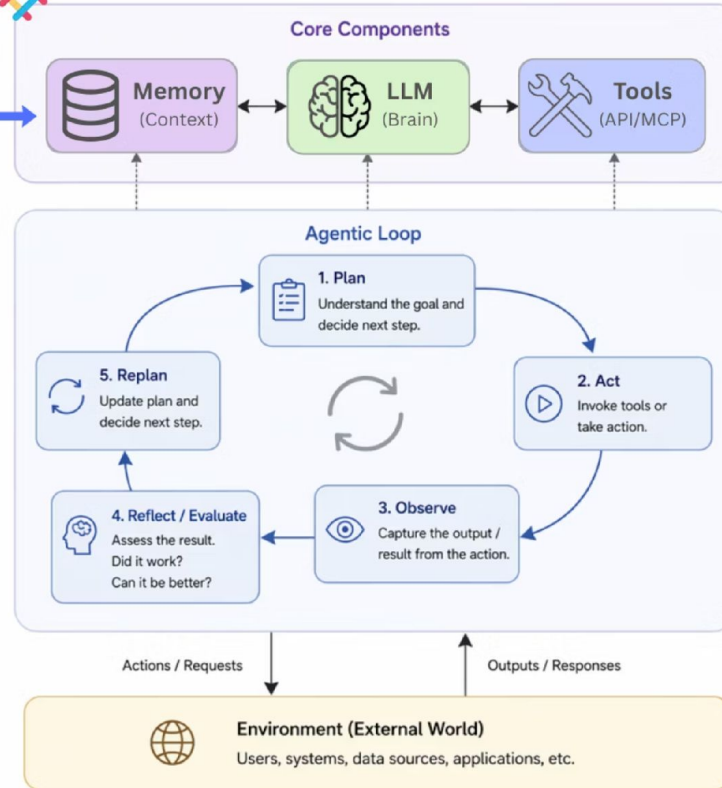


Tools (APIs/MCPs/Integrations)

How agent interacts with the outside world and performs actions - open apps, browse web, query DB



AI Agent Architecture

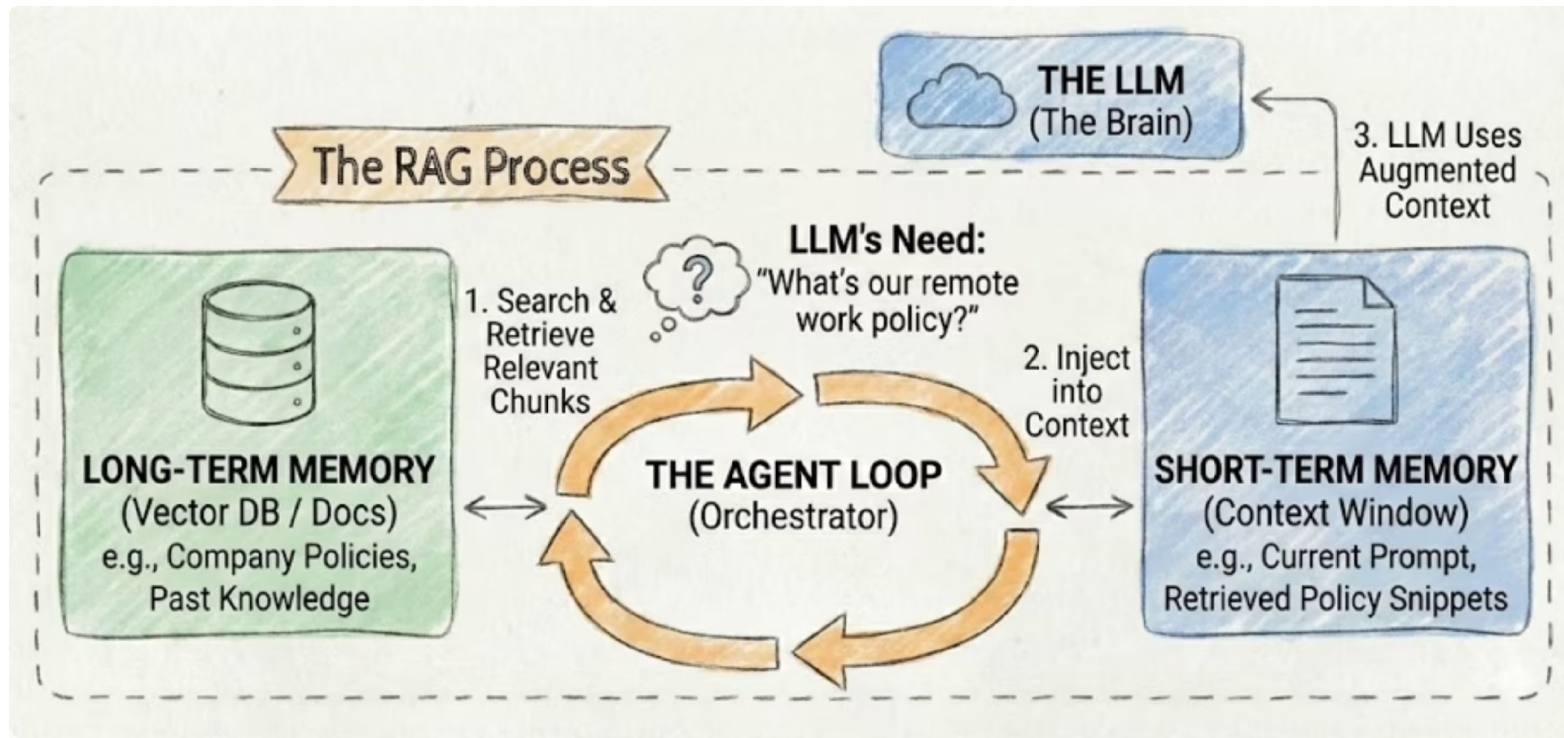


Agentic Loop

1. **Plan**: understands the goal, create plan and next steps
2. **Act**: use the model to invoke tools/take actions
3. **Observe**: Review the results of output
4. **Reflect/Evaluate**: Analyze result, how close is it to the? Accurate? Can it be improved?
5. **Replan**: If needed, update plan, replat the loop until goal or stop condition is met.

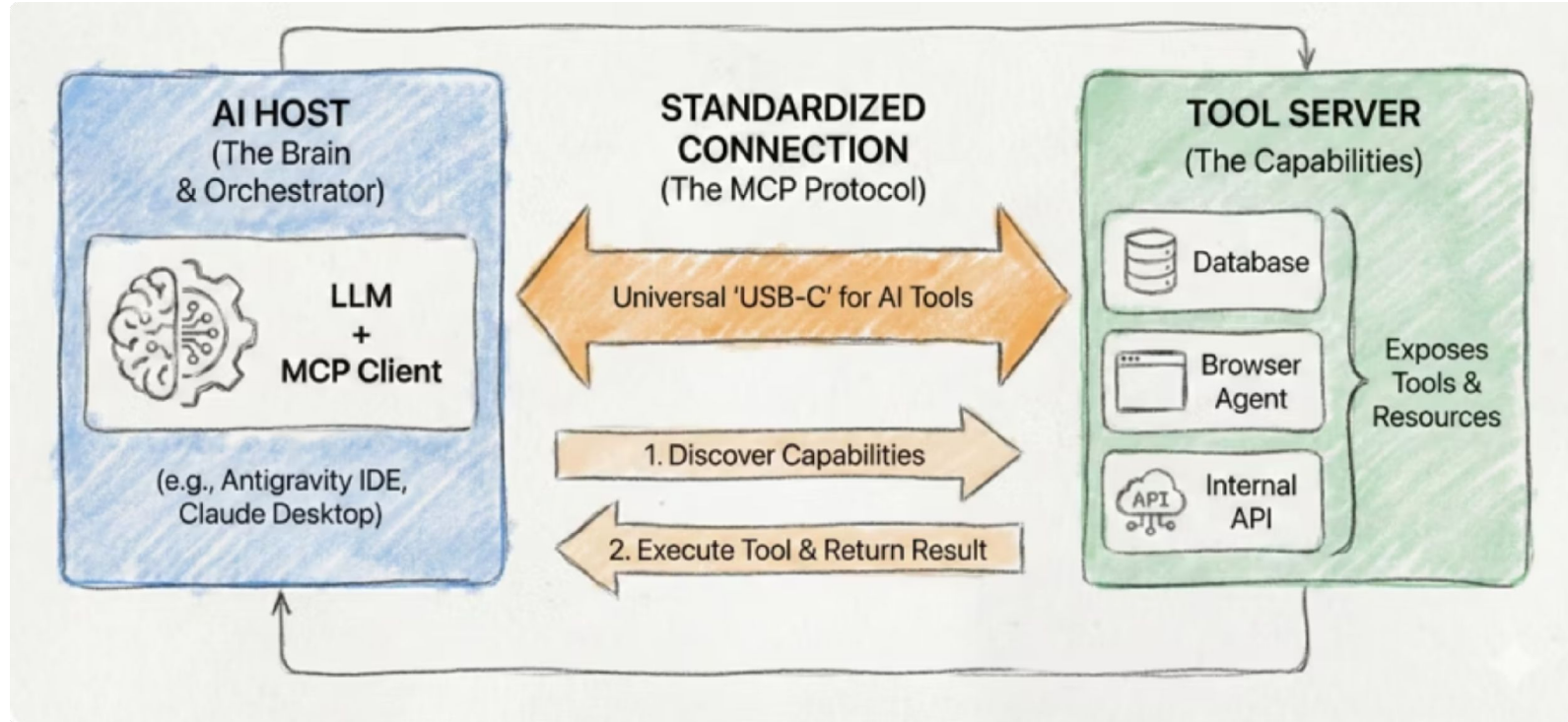
Retrieval Augmented Generation (RAG)

Moving Information From Long-Term to Short-Term Memory



Model Context Protocol (MCP)

Connecting Chatbot to Outside World



What Is OpenClaw?

#1 Open Source Project to build and deploy agents

Self-hosted gateway that connects your favorite chat apps — WhatsApp, Telegram, Discord, iMessage, Slack, Signal, and more — to AI agents that runs 24/7.

Value Proposition

Model-Agnostic tool to deploy highly flexible AI agents with custom permission boundaries and security controls.

Who is it for:

Developers and power users who wants a personal AI assistant to message from anywhere — without giving up control of their data or relying on a hosted service.



OpenClaw

THE AI THAT ACTUALLY DOES THINGS.

Features

Always-on **self-hosted** OpenClaw agents are context-aware and can take actions across your digital life—not just respond to prompts.

Channels



Discord, Slack, WhatsApp, Telegram, Microsoft Teams, Google Chat, voice call with 3P, and group chat support with a single gateway.

Multi-channel: one gateway serving messaging apps simultaneously with **persistent memory** across sessions

Auth & Model Providers



- 35+ model providers
- Subscription Authorization via OAuth (OpenAI/Anthropic)
- Custom/self-hosted provider support (ex: vLLM, SGLang, Ollama, etc)

Agent



- web search, execute code/shell, file management, check calendar at runtime with tool streaming
- multi-agent routing with isolated sessions per workspace or sender

Self-improving: can write and modify its own skills

Tools



- Browser automation, exec, sandboxing
- Web search Engines (DuckDuckGo, Exa, Firecrawl, Gemini, Grok, Ollama Web Search, Perplexity, etc)
- Cron jobs and heartbeat scheduling
- Skills, plugins, and workflow pipelines.

Media



- Images, audio, video, and documents in and out
- Shared image generation and video generation over messaging channel.
- Voice note transcription
- Text-to-speech with different digital voice providers

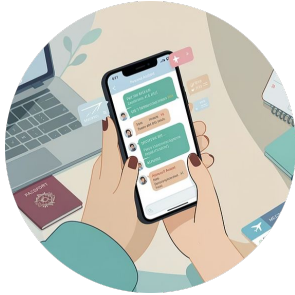
Apps and Interfaces



- WebChat and browser Control UI to manage/chat with agent
- macOS menu bar plugin
- iOS/Android pairing, Canvas, camera, screen recording, location, voice, chat, and device commands

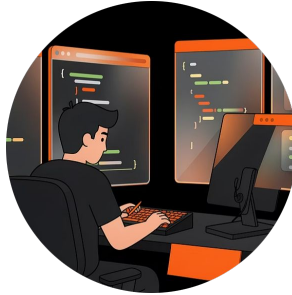
full feature details: <https://docs.openclaw.ai/concepts/features>

Use Cases You Can Build Right Now



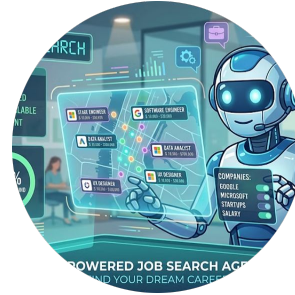
Personal Assistant

Manage health reimbursements, schedule doctor appointments, search emails, manage your calendar, book/check-in flights, research topics, custom morning briefings



Developer Workflow

Automate test runs, open & review PRs, respond/resolve defects/errors, run Claude Code loops. Monitor your release workflow and get alerts when done.



Job Search Companion

Chat with your agent that adapts to where you are in the journey - discovery, application, interview prep



Smart Home Automation

Control your smart devices, monitor air quality based on biomarkers and control smart home devices.



Security Foundations

1

NETWORK ISOLATION

lowest blast radius

- VPC · firewalls
- No internet egress by default

2

DATA FILTERING

protects context

- PII redaction
- Field-level masking before model sees data

3

TOOL SCOPING

Limits action radius

- Read-only by default
- Explicit allowlist for any write/delete access

4

RUNTIME CONTROLS

highest risk if bypass

- Approval gates
- Session timeouts
- Kill switches

Security Principles every agent needs



1 Least Privilege by Default

Every tool gets the minimum permission it needs. Read-only until proven otherwise. No admin access without explicit approval.

2 Sandbox Execution Environments

Run agents in isolated containers or VMs. Never direct access to production.

3 Human Approval Gates

For any high-risk action — writes, deletes, external API calls — require explicit human confirmation before execution.

4 Full Audit Logging

Log every thought, tool call, and output. If you can't audit it, you shouldn't run it.

Before Going Live

1 Run built-in security audit

OpenClaw ships with security scanner to flag critical misconfigs files before exposed to tools. Run it immediately after install

1

2

2 Lock file permissions on ~/.openclaw

Session transcripts, API keys, channel credentials, and your config all live under this directory. Default permissions remains open to all users on the system who can read your conversation history, access keys, and credentials.

3 Set DM policy to "pairing" on every channel

The default is pairing but it's easy to accidentally switch to open during setup. Open DMs means anyone who can reach your bot can trigger real tool calls with your agent's full permissions

3

4

4 require explicit @mention in all group chats

Without using @<botname> gating, your agent responds to every message in every group it's in. Any group member can trigger tool calls — including people you didn't intend to give access.

5 Deny dangerous tools from the start

The gateway/cron/sessions_spawn/sessions_send can read, make persistent config changes that survive restarts, change tool policies, schedule jobs after original session ends. deny these tools by default to research agents who may handle untrusted content

5

6

6 Use strong models for tool-enabled agents

Smaller/cheaper models are significantly more susceptible to prompt injection. Saving money on the model for a tool-enabled agent is a security tradeoff, not just a quality tradeoff. Don't use for agents with execution, browser, or file sys access. Use updated models for tool-enabled agents. save the rest for chat, no-tool agents only.

OpenClaw security model details: <https://docs.openclaw.ai/gateway/security>

The Claw Ecosystem

From Full-featured to Purpose-built

Variant	Positioning	Key Advantages	Best For	Resources	User type
NemoClaw (NVIDIA)	Enterprise-Grade Security Platform	Enterprise compliance & tool integration (Jira, GitHub, Slack), privacy controls, GPU acceleration	Meeting regulatory requirements	Enterprise server architecture	Enterprises
OpenClaw (OpenAI)	All-in-One Personal Assistant	Richest ecosystem, 5,000+ skills, broadest channel support	General users who want everything	~1.5GB memory, ~28MB binary	General Users
NanoClaw	Isolation-First Protect host	Container-native, OS-level isolated sandboxes, RCE protection	Security and host protection	~400MB memory, ~15MB binary	Privacy & Security
IronClaw	Security-First Protect Data	Sandboxed tools, key/password encryption, scans outbound data, prompt injection defense, narrower support	High-security deployment requirements	~50MB memory, ~8MB binary	Privacy & Security, tech early adopters
PicoClaw	Hardware-First	Embedded/low-power, runs on \$10 devices, Whisper voice integration	IoT and embedded physical AI assistants	<10MB memory, ~5MB binary	Developers & Enthusiasts
ZeroClaw	High-Performance	extreme resource efficiency and speed (<10ms startup), high concurrency	Developers, edge computing, automation	~8MB memory, ~3.4MB binary	Developers & Enthusiasts
NullClaw	Extreme Minimalist	<2ms start-up, no dependencies, edge/embedded devices	Most constrained environments	~1MB memory, <500KB binary	Developers & Enthusiasts

source: <https://nemoclaw.bot/claw-ecosystem-overview.html>

Should you use OpenClaw

 tiffusionlabs.github.io

Should you use OpenCla





OpenClaw

THE AI THAT ACTUALLY DOES THINGS.

to get started visit:

<https://docs.openclaw.ai/>

Security Threats

1

Open DMs let anyone trigger your agent

If DM policy is set to open, anyone who can message your bot can induce real tool calls — exec, browser, file access — within your agent's permissions and security policies.

2

Prompt injection via content, not senders

Malicious instructions may arrive through web search results, fetched URLs, emails, attachments, or pasted logs that your bot reads. The content itself is the attack vector — not the messenger.

3

Group chats are a wide-open by default

Anyone in a Slack or WhatsApp group with your agent can steer it. One bad actor in a shared room can inject instructions that affect shared state, devices, trigger file writes, or exfiltrate data.

4

The gateway and cron tools can rewrite your config

These tools can make persistent control-plane changes and schedule jobs that keep running after the original chat ends. If an attacker controls your agent's input, they can make changes after system restarts.

5

Session transcripts store everything on disk

Every conversation — including tool outputs, API responses, pasted secrets, and file contents — are written to the sessions json files. Any process or user with filesystem access can read it all.

6

Plugins run in-process with full gateway access

OpenClaw plugins run directly inside the gateway process and a malicious plugin has full access to your credentials, sessions, and tools. Only install plugins from trusted sources.

7

Weak models are a security liability

Smaller, cheaper models are generally more susceptible to tool misuse and higher risk to prompt injection to hijack instructions.



Isolating Agents from Private Information

What NOT to expose to any agent

PII, credentials, and secrets: API keys, passwords, tokens, SSNs — never in agent context

Unscoped database or API access: agents should never see a full production DB connection

Internal system topology details: network maps, service names, infrastructure layout

Proprietary training data or IP: anything you wouldn't want in a prompt leak

Isolation strategies

Context filtering: Strip sensitive fields before passing data to the agent — never pass raw DB records or API responses.

Separate environments: Dev, staging, and prod agents never share state. One compromised agent stays contained.

Token-level access: Use short-lived, scoped tokens (temp credential with limited actions) — not service account keys (permanent credential/master pwd). Rotate & Revoke them fast.